Qué hace cada función del proyecto

Recorrido por los scripts de la práctica 3: el recolector de Renfe, los generadores del mapa, la aplicación Flask, el SQL y el JavaScript de la página.

Práctica 3 · trenes.py · generar_lineas.py · generar_red.py · app.py · init.sql · index.html

api/trenes.py

21 funciones y una clase: baja los feeds y publica por MQTT.

api/generar_lineas.py · generar_red.py

10 funciones que dibujan las vías del mapa.

app.py

27 funciones: 18 de apoyo y 9 rutas. Sincroniza, consulta y sirve la API.

init.sql y ESQUEMA_SQL_ADICIONAL

Tablas, vistas y la función trenes_en().

templates/index.html

43 funciones JavaScript del mapa y la línea de tiempo.

Cómo se reparte el trabajo entre los scripts

Fichero	Lenguaje	Papel	Se ejecuta
`api/trenes.py`	Python	Recolector: Renfe → MQTT. Cliente MQTT propio.	En el Mac, en una ventana de Terminal
`api/generar_lineas.py`	Python	Crea `static/lineas.geojson` con las líneas de Cercanías.	Una vez a mano
`api/generar_red.py`	Python	Crea `static/red.geojson` desde OpenStreetMap.	Una vez a mano
`app.py`	Python · Flask	Sincroniza Influx → Postgres y sirve la web y la API.	Contenedor `p3-flask`
`init.sql` + `ESQUEMA_SQL_ADICIONAL`	SQL	Esquema, vistas y función de consulta por instante.	Al crear el volumen / al arrancar Flask
`templates/index.html`	HTML · JS	Página con mapa Leaflet, línea de tiempo y leyenda.	En el navegador
`telegraf.conf`	TOML	Cuatro entradas MQTT hacia InfluxDB.	Contenedor `p3-telegraf`

Datos estáticos: los ficheros GTFS

Dos estructuras auxiliares: `DatosGtfsEstaticos` agrupa lo cargado y `UltimaPosicionPublicada` recuerda la última posición enviada de cada tren.

Función	Qué hace
`descargar_url(url, como_bytes)`	Descarga una URL con un User-Agent propio y 30 s de espera. Devuelve el JSON ya leído o, con `como_bytes`, los bytes del ZIP.
`leer_filas_de_un_csv_dentro_del_zip(zip, csv)`	Recorre un CSV del ZIP (UTF-8 con BOM) y entrega cada fila como diccionario con los espacios recortados.
`abrir_gtfs_descargandolo_si_esta_viejo_o_no_existe(url, ruta)`	Usa la copia de `api/.cache`. Si falta o tiene más de 7 días, la descarga otra vez. Devuelve el `ZipFile`.
`calcular_primera_y_ultima_parada_de_cada_viaje(zip)`	Lee `stop_times.txt` y guarda, por viaje, la parada con menor y con mayor `stop_sequence`: de dónde a dónde va.
`cargar_gtfs_estaticos_de_cercanias_y_larga_distancia()`	Carga los dos GTFS y devuelve `DatosGtfsEstaticos`: núcleo por viaje, tipo de servicio por número de tren, nombres y coordenadas de paradas y extremos de cada viaje. Si uno falla, avisa y sigue.

Distancia y cliente MQTT hecho a mano

Función	Qué hace
`calcular_distancia_en_km_entre_dos_coordenadas(…)`	Fórmula de haversine con el diámetro de la Tierra (12 742 km). Sirve para estimar la velocidad.
`ClienteMqttMinimo.__init__(identificador)`	Guarda el identificador del cliente y deja la conexión en `None`.
`codificar_texto_con_su_longitud(bytes)`	Antepone dos bytes con la longitud, como exige MQTT para topics e identificadores.
`construir_paquete(tipo, cuerpo)`	Monta el paquete: tipo, longitud restante en formato variable (7 bits por byte) y cuerpo.
`conectar_con_el_broker()`	Abre el socket a `MQTT_HOST:MQTT_PORT`, envía CONNECT (MQTT 3.1.1) y comprueba el CONNACK.
`publicar(topic, valor)`	Envía un PUBLISH sin confirmación. Si la conexión falla, la reabre y reintenta una vez; si no, lanza `ConnectionError`.

Lectura de los feeds de Renfe

Función	Qué hace
`leer_retrasos_por_viaje_y_anotar_los_viajes_cancelados(fichero, ids)`	Descarga `trip_updates`. Devuelve el retraso por `tripId` (el del viaje o el de su primera parada) y añade a la lista los viajes `CANCELED`.
`leer_trenes_de_cercanias(núcleo_por_viaje, cancelados)`	Lee posiciones y retrasos de Cercanías. Genera un diccionario por tren con código, línea (de la etiqueta), núcleo y categoría.
`leer_trenes_de_larga_distancia(tipo_por_tren, cancelados)`	Igual con los feeds `_LD`. Ignora trenes sin tipo de servicio en el GTFS; la hora es la de recepción; la categoría sale de una tabla (AVE → Alta velocidad…).
`construir_mensaje_de_viaje_cancelado(id, es_ld, gtfs)`	Larga distancia: marca, número, origen, destino y coordenadas del origen desde el GTFS. Cercanías: saca núcleo, número y línea del `tripId` con una expresión regular.
`reunir_trenes_y_viajes_cancelados_de_los_dos_feeds(gtfs)`	Llama a los dos lectores. Si un feed no responde, avisa y sigue con el otro.

Qué se publica y qué se descarta

Un tren de Cercanías sin posición nueva en 10 minutos deja de contar como activo.

Función	Qué hace
`tren_sigue_en_la_misma_posicion_que_la_ultima_publicada(…)`	Evita duplicados. Cercanías compara el timestamp; larga distancia compara coordenadas y republica cada 120 s aunque no se mueva.
`estimar_velocidad_en_kmh_desde_la_ultima_posicion(…)`	Distancia entre dos posiciones dividida por el tiempo (entre 10 y 300 s). Se descarta si supera la velocidad máxima creíble de su categoría.
`construir_mensaje_de_posicion_de_tren(…)`	Arma el JSON: tren, línea, núcleo, categoría, viaje, estado, parada, lat, lon, hora en ms, retraso y velocidad si la hay.
`publicar_las_posiciones_que_han_cambiado(…)`	Recorre los trenes, marca los activos, salta los repetidos y publica el resto en `renfe/trenes`. Devuelve cuántas son nuevas y los activos.

Estadísticas, limpieza y bucle principal

Función	Qué hace
`calcular_retraso_medio_y_cuantos_van_retrasados(lista)`	Media de los retrasos conocidos y cuántos llegan a 300 s o más.
`publicar_estadisticas_globales(cliente, activos)`	Publica `renfe/stats/trenes_activos`, `retraso_medio_s` y `con_retraso`.
`publicar_estadisticas_por_nucleo(cliente, activos)`	Publica un mensaje `renfe/nucleos` por núcleo con trenes, retraso medio y con retraso.
`olvidar_los_trenes_no_vistos_desde_hace_tiempo(…)`	Borra de la memoria los trenes que no se publican desde hace una hora.
`publicar_los_viajes_cancelados_y_olvidar_los_que_ya_no_estan(…)`	Publica cada cancelado en `renfe/cancelados` al verlo y cada 10 min mientras siga; olvida los que desaparecen del feed.
`consultar_renfe_y_publicar_en_mqtt_para_siempre()`	Bucle principal: carga los GTFS y, cada 30 s, reúne trenes, publica y muestra una línea de resumen. Captura errores y duerme el tiempo que falte.

`api/generar_lineas.py`

Dibujar las líneas de Cercanías con sus colores

Función	Qué hace
`calcular_distancia_de_un_punto_a_un_segmento(p, a, b)`	Proyecta el punto sobre el segmento a–b y devuelve la distancia al punto más cercano.
`simplificar_trazado_con_ramer_douglas_peucker(puntos, tolerancia)`	Simplifica una polilínea: conserva extremos y los puntos más lejanos de la recta hasta que ninguno supere la tolerancia.
`descargar_gtfs_de_cercanias_si_no_esta_en_la_cache()`	Descarga el ZIP (~16 MB) solo si aún no está en `api/.cache`.
`leer_rutas_y_puntos_de_cada_trazado_del_gtfs()`	Lee rutas, la ruta de cada trazado y los puntos de `shapes.txt`, ignorando los `_INV` (sentido contrario).
`construir_geojson_de_las_lineas_de_cercanias()`	Ordena y simplifica cada trazado y crea un `Feature` con línea, núcleo y color de Renfe.
`guardar_geojson_de_las_lineas_de_cercanias()`	Orquesta todo y escribe `static/lineas.geojson` compacto, con un resumen de líneas, puntos y KB.

`api/generar_red.py`

Dibujar la red ferroviaria desde OpenStreetMap

Reutiliza `simplificar_trazado_con_ramer_douglas_peucker` de `generar_lineas.py`, con una tolerancia mayor (0,0008°).

Función	Qué hace
`descargar_vias_de_una_cuadricula_desde_overpass(cuadricula)`	Pregunta a Overpass por las vías principales (`railway=rail` , `usage=main`) de España en un rectángulo. Reintenta hasta 6 veces con espera creciente.
`descargar_y_simplificar_las_vias_de_toda_la_peninsula()`	Recorre 12 cuadrículas, evita vías repetidas, simplifica con Ramer–Douglas–Peucker y marca las de alta velocidad.
`construir_geojson_con_vias_convencionales_y_de_alta_velocidad(vias)`	Agrupa los trazados en dos `MultiLineString` : convencional y alta velocidad.
`guardar_geojson_de_la_red_ferroviaria()`	Descarga, construye y escribe `static/red.geojson` compacto, e imprime tramos y tamaño.

Hablar con InfluxDB y localizar identificadores

Función	Qué hace
`consultar_influxdb(consulta)`	Lanza una consulta InfluxQL por HTTP con credenciales y tiempo en ms. Si Influx devuelve un error, lo eleva; si no, entrega las series.
`convertir_series_de_influxdb_a_diccionarios(series)`	Convierte columnas y valores en un diccionario por punto.
`convertir_milisegundos_a_fecha(ms)`	Pasa milisegundos desde 1970 a un `datetime` en UTC.
`calcular_milisegundos_desde_los_que_leer(último)`	Empieza 120 s antes del último dato guardado para solapar; la primera vez lee las últimas 2 h.
`construir_diccionario_de_claves_a_ids(cursor, sql)`	Ejecuta un SELECT y devuelve `{clave: id}`.
`cargar_ids_de_nucleos_lineas_y_trenes(cursor)`	Carga los tres diccionarios para no consultar la base por cada punto.
`obtener_o_crear_id_del_tren(cursor, ids, punto)`	Busca núcleo → línea → tren y los inserta si faltan. Devuelve el id del tren.

Copiar de InfluxDB a PostgreSQL cada 30 s

Función	Qué hace
`copiar_posiciones_de_influxdb_a_postgres(cursor)`	Lee los puntos nuevos de `trenes`, descarta los sin posición, crea tren y parada si faltan e inserta en `posiciones` sin duplicar.
`copiar_viajes_cancelados_de_influxdb_a_postgres(cursor)`	Agrupa por (tren, viaje) con primera y última vez vista y hace *upsert* en `cancelaciones`.
`crear_tablas_vistas_e_incidentes_que_faltan()`	Ejecuta `ESQUEMA_SQL_ADICIONAL` (cancelaciones, vistas, `trenes_en`, incidentes) y siembra los incidentes a mano.
`calcular_retrasos_por_hora_y_por_dia_y_borrar_datos_antiguos(cursor)`	Recalcula `retrasos_hora` y `retrasos_dia` y borra posiciones y cancelaciones de más de 48 h.
`sincronizar_influxdb_con_postgres()`	Con candado y en una sola transacción: copia, agrega y anota en `sincronizaciones`. Si falla, deshace y anota el error.
`sincronizar_para_siempre_en_segundo_plano()`	Hilo que crea lo que falte la primera vez y sincroniza cada 30 s sin morir ante un error.

Consultas a PostgreSQL y conversión a GeoJSON

Función	Qué hace
`ejecutar_consulta_sql(sql, argumentos)`	Abre conexión, ejecuta y devuelve las filas como lista de diccionarios.
`calcular_resumen_en_instante(instante)`	Una consulta sobre `trenes_en(t)`: trenes, en movimiento, parados, retraso medio, con retraso, cancelados, núcleos, líneas y posiciones.
`leer_instante_pedido_en_la_url()`	Lee `?t=` en ISO 8601; sin él, ahora. Responde 400 si no es una fecha válida y asume UTC si falta la zona.
`convertir_trenes_en_instante_a_geojson(instante)`	Llama a `trenes_en` y devuelve una `FeatureCollection` de puntos con marca, retraso, parada y movimiento.
`convertir_cancelados_con_posicion_en_instante_a_geojson(instante)`	Cancelados con coordenadas de origen vistos en los 15 min anteriores al instante.

app.py · 4 de 4

Las nueve rutas de Flask

Ruta	Función	Qué devuelve
/	mostrar_pagina_principal	La página con las cifras y el mapa
/api/trenes	devolver_trenes_del_instante_en_geojson	Trenes del instante en GeoJSON
/api/cancelados	devolver_cancelados_del_instante_en_geojson	Cancelados con posición en GeoJSON
/api/instante	devolver_estado_del_mapa_en_el_instante	Trenes, cancelados y resumen en una sola petición; la usa la línea de tiempo
/api/rango	devolver_cobertura_del_historico	Primer dato, ahora y nº de posiciones por cubo de 10 min
/api/incidentes	devolver_incidentes_en_geojson	Incidentes a mano con trenes, texto y fuentes
/api/resumen	devolver_resumen_del_instante	Solo las cifras del instante
/sync	sincronizar_manualmente	Fuerza una sincronización; redirige a / o responde 502
/health	comprobar_salud_del_servicio	{"estado": "ok"} si Postgres responde

`init.sql · ESQUEMA_SQL_ADICIONAL`

Las vistas y la función que dan forma a los datos

Función	Qué hace
`trenes_en(ts)`	Función SQL. Para cada tren, su última posición en los 15 min anteriores a `ts`, con `en_movimiento`: no es `STOPPED_AT` y se ha movido más de ~30 m en 3 min.
`v_trenes_ahora`	Es `trenes_en(now())`. Alimenta las tablas y Grafana.
`v_lineas · v_nucleos`	Última hora: trenes, retraso medio y máximo y % de puntualidad (retraso < 300 s).
`v_cancelados_ahora`	Cancelaciones vistas en los últimos 15 min con marca, núcleo y coordenadas de origen.
`retrasos_hora · retrasos_dia`	Tablas agregadas que rellena Flask con `INSERT … ON CONFLICT DO UPDATE`.
`índice en posiciones(observado_en)`	Acelera la búsqueda por tramo de tiempo que usa la línea de tiempo.

JavaScript · utilidades y recarga automática

Función	Qué hace
`leerDelAlmacenamiento(almacén, clave)`	Lee de `localStorage` o `sessionStorage` sin romper si el navegador lo bloquea.
`guardarEnElAlmacenamiento(almacén, clave, valor)`	Guarda con la misma protección ante errores.
`escaparHtml(texto)`	Sustituye `& < > " '` para insertar texto en popups sin riesgo.
`obtenerElemento(id)`	Atajo para `document.getElementById`.
`programarLaRecargaAutomatica()`	Programa recargar la página cada 30 s, salvo que esté desactivada o se mire el pasado.
`desactivarLaRecargaMientrasSeVeElPasado()`	Apaga la recarga cuando el usuario se mueve por la línea de tiempo.
`leerVistaGuardadaDelMapa()`	Recupera centro y zoom guardados en `sessionStorage` para no perder la vista al recargar.

JavaScript · mapa base e iconos de tren

Función	Qué hace
`crearCapaDeTeselasEsri(servicio, opciones)`	Crea una capa Leaflet con teselas de Esri.
`crearMapaBaseGris(tono, conEtiquetas)`	Mapa base gris, claro u oscuro, con o sin nombres de lugares.
`esMarcaDeCercanias(marca)`	Verdadero para Cercanías y Rodalies.
`colorSegunElRetraso(segundos)`	Gris sin dato; verde < 1 min; amarillo; naranja ≥ 5 min; rojo ≥ 10 min.
`obtenerMarcaDelTren(propiedades)`	Deduce la marca (AVE, AVLO, ALVIA, MD, Rodalies…) desde categoría, núcleo y tipo de servicio.
`obtenerColorDeMarcaDelTren(marca, propiedades)`	Color de la marca; en Cercanías, el de su línea según `lineas.geojson`.
`crearSvgDeTren(marca, opciones)`	Dibuja la silueta SVG de cada marca, con estela, punto de retraso y estilo parado o en marcha.
`crearIconoDeTrenParaElMapa(…)`	Convierte el SVG en icono Leaflet con su etiqueta corta.
`mostrarEtiquetasDeTrenesSoloSiElMapaEstaAcercado()`	Muestra las etiquetas solo desde el zoom 8.

JavaScript · color, popups, incidentes y cancelados

Función	Qué hace
`marcarElBotonDelModoDeColorActivo()`	Resalta el botón Marca o Retraso.
`cambiarElModoDeColorDelIcono(modo)`	Guarda el modo y redibuja todos los iconos.
`crearTituloDelPopupDeUnTren(marca, propiedades)`	Título del popup: marca y número, y núcleo en Cercanías.
`crearPopupDeUnTren(marca, propiedades, color)`	Popup con movimiento, retraso, velocidad, parada y hora de la posición.
`crearHtmlDeLaListaDeTrenesDeUnIncidente(trenes)`	Lista de trenes implicados, con marca, número, trayecto y viajeros.
`dibujarLosIncidentesQueYaHanOcurrido()`	Pone el marcador amarillo de cada incidente solo si su hora es anterior al instante mostrado.
`agruparCanceladosPorEstacionDeOrigen(geojson)`	Agrupa los cancelados por la estación de la que salían.
`dibujarLosViajesCanceladosAgrupadosPorEstacionDeOrigen(geojson)`	Marcador rojo «✕ N» por estación y popup con los viajes.

JavaScript · dibujar todo lo de un instante

Función	Qué hace
`actualizarLasCifrasDeArriba(resumen)`	Escribe las seis cifras de la cabecera.
`dibujarLosTrenesDelInstante(geojson)`	Vacía las capas, crea un marcador por tren y lo manda a su capa (marca en movimiento o parados). Devuelve los contadores.
`ajustarElMapaParaVerTodosLosTrenesSiNoHayVistaGuardada(geojson)`	La primera vez, encuadra todos los trenes salvo que haya vista guardada.
`actualizarElControlDeCapasConLosContadoresDelInstante(…)`	Rehace el selector de capas con el número de trenes de cada marca y de cancelados.
`dibujarTodoLoQueDependeDelInstante(estado)`	Coordina: trenes, cancelados, incidentes, cifras y capas de vías la primera vez.
`formatearFechaYHoraCorta(ms)`	Formatea fecha y hora en español: «mié 28/10, 16:30».

templates/index.html · 5 de 6

JavaScript · línea de tiempo y reproducción

Función	Qué hace
`actualizarLosTextosDeLaLineaDeTiempo(n)`	Rotula el instante («Ahora (en directo)» o la fecha) y avisa si no hay datos.
`dibujarElHistogramaDeCobertura()`	Dibuja barras SVG con el nº de posiciones por cubo de 10 min sobre el deslizador.
`ajustarElRangoDelDeslizadorSegunLaVentanaElegida()`	Fija mínimo y máximo según 1 h, 6 h, 24 h o todo el histórico.
`pedirAlServidorElEstadoDelMapaEnUnInstante(ms)`	Pide `/api/instante?t=…` y dibuja. Si hay una petición en curso, guarda solo la última pedida.
`alCambiarElInstanteSeleccionado()`	Decide si se está en directo (vuelve a recargar) o en el pasado (apaga la recarga) y pide el estado.
`alternarLaReproduccion()` · `pausarLaReproduccion()`	Cada segundo avanza ×60, ×300 o ×900 s de historia; se detiene al llegar al final.
`saltarEnLaLineaDeTiempo(ms)`	Mueve el deslizador ±10 minutos sin salirse del rango.

JavaScript · la leyenda del mapa

Función	Qué hace
`obtenerTextoExtraDeLaMarcaEnLaLeyenda(marca)`	Añade notas: «color de la línea» en Cercanías e «Iryo: solo en incidentes».
`crearFilasDeLeyenda(iconosYTextos)`	Convierte pares icono–texto en filas HTML.
`crearGruposDeMarcasDeLaLeyenda(opciones)`	Agrupa las marcas por familia: alta velocidad, larga distancia, media distancia y Cercanías.
`laLeyendaDebeEstarAbierta()`	Abierta por defecto en pantallas de 900 px o más, o según lo guardado.
`dibujarLaLeyenda()`	Pinta la caja plegable con iconos, grupos y la escala de colores del retraso.

FIN

Los dos recorridos, a mano: la presentación del proyecto y esta de funciones.

Flask

Mapa y línea de tiempo

`localhost:5002`

Grafana

Dashboard Trenes Renfe

`localhost:3001`

pgAdmin

Base PostgreSQL

`localhost:5050`